A Formal Verification approach for mitigating Neural Network approximations of Acas-Xu

Martin Boniol
martin.boniol@isae-supaero.fr
ISAE-SUPAERO & ONERA
Toulouse, 31055, France

Julien Brunel
ONERA
Toulouse, 31055, France
julien.brunel@onera.fr

Jean-Baptiste Chaudron
ISAE-SUPAERO
Toulouse, 31055, France
jean-baptiste.chaudron@isae-supaero.fr

Christophe Garion
ISAE-SUPAERO
Toulouse, 31055, France
christophe.garion@isae-supaero.fr

Xavier Thirioux
ISAE-SUPAERO
Toulouse, 31055, France
xavier.thirioux@isae-supaero.fr

Abstract

Ensuring air traffic safety relies on highly sophisticated collision avoidance systems. This work focuses on ACAS-Xu, the collision avoidance system designed for unmanned aircraft. Its decision logic is implemented through Look-Up Tables (LUTs) generated from Markov Decision Processes (MDP), whose large size poses challenges for UAV deployment. Recent studies have proposed Artificial Neural Networks (ANNs) as lightweight approximations of these tables, yet understanding their limitations and unsafe behaviors remains an open problem. In this work, we first provide a detailed analysis and clarification of ACAS-Xu's operational behavior, addressing ambiguities in existing literature. Then, we introduce a framework for identifying and characterizing weaknesses in ANN-based approximations of ACAS-Xu. The method relies on a structured behavioral specification—covering nominal, single-intruder, and multi-intruder scenarios—to detect regions where the neural networks deviate from the certified logic. This analysis enables the systematic localization of unsafe regions and supports the construction of an efficient reduced Safety Net, ensuring robust advisory generation even in complex encounter conditions.

Keywords

ACAS-Xu, Neural Networks, Formal Verification, Safety-Critical Systems, UAV

ACM Reference Format:
Martin Boniol, Julien Brunel, Jean-Baptiste Chaudron, Christophe Garion, and Xavier Thirioux. 2026. A Formal Verification approach for mitigating Neural Network approximations of Acas-Xu. In . ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Ensuring flight safety in increasingly crowded airspaces requires sophisticated anti-collision systems. For decades, the Traffic Collision Avoidance System (TCAS) has served as a cornerstone of civil aviation, significantly reducing the risk of mid-air collisions. Yet, as air traffic grows and operational contexts diversify, traditional TCAS architectures have reached their practical and technological limits [8, 16]. To overcome these limitations, the Airborne Collision Avoidance System X (ACAS-X) was introduced as a next-generation framework, employing advanced probabilistic modeling and decision-making techniques. ACAS-X extends collision avoidance capabilities beyond commercial airliners to General Aviation aircraft and Unmanned Aerial Vehicles (UAVs), through its variants ACAS-Xa and ACAS-Xu [6]. This work focuses specifically on ACAS-Xu, the variant designed for unmanned aircraft. Unlike ACAS-Xa, which is limited to vertical maneuvers, ACAS-Xu supports both horizontal and vertical avoidance strategies, extending the decision problem into a fully three-dimensional space. While this design enables a richer and more flexible collision avoidance logic, it also introduces additional complexity that makes the system more difficult to analyze, verify, and implement [4]. At the core of ACAS-Xu lies a real-time decision logic engine that continuously evaluates possible future aircraft trajectories to select the safest maneuver. This logic is encoded through Look-Up Tables (LUTs), which store precomputed optimal actions derived offline using Markov Decision Processes (MDP) [12, 13]. Consulting these LUTs allows ACAS-Xu to make rapid decisions without performing costly onboard computations. However, the practical deployment of these LUTs on UAVs raises significant challenges. Their large size, necessary to capture the diversity of flight conditions, aircraft states, and sensor uncertainties, makes them difficult to store and operate within the limited hardware resources of drones.

To address this issue, recent research has proposed replacing LUTs with Artificial Neural Networks (ANNs) that approximate the ACAS-Xu decision logic [3, 9]. This approach promises drastic reductions in memory requirements while preserving real-time decision capability. Various ANN architectures and training strategies have been explored to replicate or enhance ACAS-Xu performance.

However, before such neural approximations can be confidently integrated into operational systems, it is essential to understand their limitations and failure modes rather than only verifying their correctness. Indeed, previous studies [1, 2, 3] have shown that the existing neural networks used for ACAS-Xu compression can exhibit unsafe behaviors under certain encounter conditions. As we are dealing with approximation-based architectures, it is inherent that some cases will deviate from the reference truth tables (LUT). In this context, our work proposes a systematic framework aimed at analyzing the weaknesses of neural networks approximating the ACAS-Xu logic and identifying the regions where mitigation means should be applied. Building upon the hybrid certification approach of [3], our framework extends the analysis to a broader behavioral level by relying on functional requirements derived from the ACAS-Xu LUT, expressed as domain-specific properties that also cover complex multi-intruder scenarios. This property-driven assessment makes it possible to detect general weaknesses in the approximation and to refine the analysis over specific LUT regions where deviations occur. Furthermore, by leveraging efficient formal solvers, we ensure rigorous verification of these properties, providing both behavioral insight and formal assurance regarding the system's performance under challenging encounter conditions.

The paper is organized as follows. Sections 2 and 3 introduce the real ACAS-Xu system and the neural network architectures approximating its LUTs. Section 4.1 presents the proposed behavioral specification, while Section 4.2 details the verification results and identification of unsafe regions. A focused LUT analysis and the construction of a reduced Safety Net are discussed in Sections 4.3 and 4.4, respectively. Finally, Section 6 concludes and outlines perspectives for future work.

2 Overall operations of the ACAS-Xu

ACAS-Xu serves as a last-resort safety mechanism to prevent midair collisions. It operates autonomously and independently of both the aircraft's navigation systems and any ground-based air traffic control services [4]. The system provides two types of alerts :

- Traffic Advisories (TAs): These assist pilots in visually identifying nearby aircraft and prepare them for potential resolution advisories.
- Resolution Advisories (RAs): These provide pilots with recommended avoidance maneuvers. A RA indicates to tell the pilot the range of vertical or horizontal rates within which the aircraft should be flown to prevent a collision. When both aircraft are equipped with ACAS, the systems communicate to coordinate complementary resolution instructions.

2.1 Horizontal Decision-Making in ACAS-X

In this work, we will focus exclusively on RA-type alerts and will consider only horizontal avoidance instructions. The selection of resolution maneuvers for vertical and horizontal avoidance is conducted independently.

Horizontal decision-making is done using LUTs to choose which action to send to pilots, depending on the state of the problem, among the following and shown in Figure 1:

- Clear of Conflict (COC)
- Weak Left (WL)
- Weak Right (WR)
- Strong Left (SL)
- Strong Right (SR)

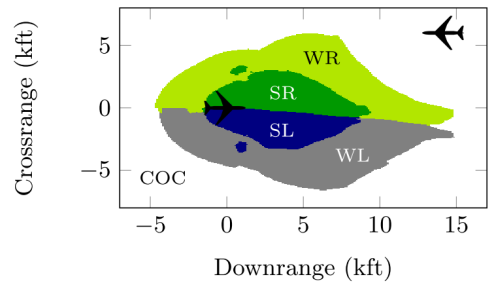

Figure 1: ACAS-Xu horizontal advisories [10]

These LUTs are huge tables that reference nearly 90 million scenarios of encounters between two aircrafts on each line. These scenarios are defined by flight parameters (aircraft speeds, trajectories, distances, etc. as shown in Figure 2) For each of these scenarios a series of costs is associated; the costs of each possible action to do in these situations, then the action is selected by choosing the one with the lowest cost.

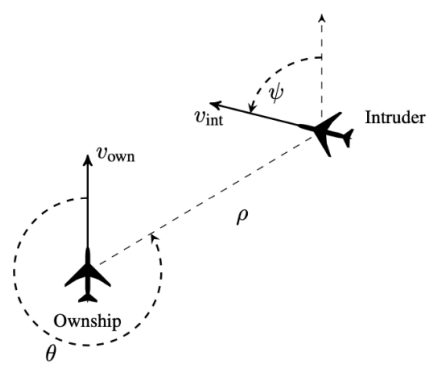

Figure 2: ACAS-Xu geometry [10]

In a horizontal collision avoidance situation, the system queries the LUT using the problem state parameters to retrieve the cost values for each possible action. Then, the final action to advise to the pilot is selected by incorporating a *lookahead* mechanism: a series of action predictions is made over a defined period. During the forecast of the aircraft's future behavior, multiple queries to the LUT are performed to determine the best possible actions over the

lookahead horizon (choosing the action with the minimum cost at each step). Finally, the chosen action aligns with the direction of the first action in the lookahead sequence and involves the strongest turn in that direction.

τ	θ	ψ	V_own	V_int	ρ	Cost_COC	Cost_WL	Cost_WR	Cost_SL	Cost_SR
0.0	-2.82	-2.67	834.1	938.0	499.0	19942	19811	19825	19739	19754

Figure 3: Example of 1 line from LUT (a_prev = COC)

For example: in Figure 3, for the scenario given by the six parameters (green variables), the selected action would be the one with the lowest cost from the 5 possible ones (red variables): **WL**. But with the *lookahead* mechanism the system will simulate the next state of the problem reach by following this advisory (and supposing the intruder is going straight) and calculate the new advisory to follow for a certain amount of time. If one of the next states during this time leads to a **SL** advisory, the real chosen action to send to the pilot will be **SL** instead of the first **WL**.

The *lookahead* mechanism is complex and not enough detailed in the literature. For the rest of this work, do not consider this mechanism in order to simplify the study. The chosen action advices by the system will be the one with the smallest cost at the first LUT query.

2.2 Multi-intruders avoidance collision

In some situations, an aircraft may face potential collisions with multiple intruders simultaneously. In such cases, the system must be able to determine the optimal maneuver to effectively avoid all threats.

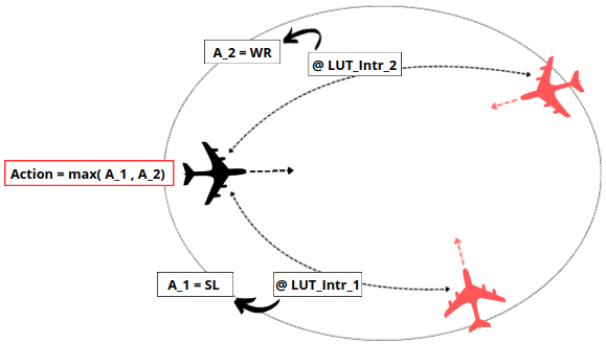

Figure 4: ACAS-Xu Multi-intruders example

The decision-making process for multi-intruder scenarios follows the same principles as those used for single-intruder avoidance. The system evaluates potential actions independently for each intruder by computing an individual action selection based on the LUT. To determine the final advisory action, the system compares the computed actions for each intruder and selects the one with the highest cost. This ensures that the maneuver chosen is the safest option, effectively mitigating the most critical collision risk while still accounting for all threats. An example of such situation is given in Figure 4.

2.3 Cooperative-flight

In case of a cooperative encounter, the two aircraft communicate with each other and exchange the actions taken, sending Horizontal Resolution Advisory Complement (HRC) or Vertical Resolution Advisory Complement (VRC) messages. As a result, the aircraft coordinate their actions to facilitate avoidance maneuvers.

Before executing the chosen action, the system first checks if an HRC or VRC message has been received. If no message is received, the selected action is executed. Otherwise, a new complementary action must be chosen.

Several scenarios arise:

- If an aircraft detects the intruder first, it becomes the master. It then decides on the action to take (according to the action selection procedure detailed later). Once the action is chosen, it communicates this decision to the intruder via an HRC or VRC message and executes the action. It does not receive any HRC or VRC alerts.
- If both aircraft make a decision about their actions before receiving any alerts, they both proceed with their chosen actions. If, at this point, they realize that their actions are incompatible, one of them must change its trajectory. The "master/slave" decision is based on the aircraft registration. The slave must then adjust its trajectory to avoid the master.

Aircraft coordination occurs across all ACAS and TCAS systems, but only within shared action plans. For example, coordination is possible between an ACAS-Xa system and an ACAS-Xu system, but only on the vertical plane as ACAS-Xa does not provide horizontal avoidance actions.

3 ACAS-Xu Compression

The LUTs used in ACAS-Xu are extremely large, with a total size of approximately 8 GB for horizontal avoidance alone. Since there are five separate LUTs, each corresponding to a different previously executed maneuver, the storage demands are significant. Additionally, vertical avoidance requires its own set of LUTs, increasing further memory requirements.

This raises a crucial challenge: how to efficiently store and deploy ACAS-Xu on aircraft, particularly drones, where computational resources and storage capacity are more limited ?

To address this issue, the research community has proposed to replace LUT-based decision-making with neural networks [10]. Instead of searching large LUTs for action costs, the system would rely on trained neural networks to approximate the same decision process. These networks would be trained using LUT data, allowing them to replicate ACAS-Xu behavior while significantly reducing storage requirements.

Several neural network architectures and training strategies have been proposed to replace LUT-based decision-making in ACAS-Xu, with the goal of preserving the same level of reliability and

accuracy. However, these approximations are inherently imperfect, and ensuring safe behavior under all operational conditions remains a major challenge. The key objective is therefore to identify their weaknesses and unsafe regions, so as to guide the application of mitigation strategies that can enhance the overall robustness of the system.

3.1 Compression into Neural Networks

The first networks we consider are those developed in [9]. The objective was to use a deep neural network based approach to map input variables to collision avoidance advisories. Instead of searching large LUTs, the system assigns a score to each advisory, with the lowest score corresponding to the optimal avoidance action.

For the remaining of the paper, these networks are denoted by "**Julian_networks**" (or **J_nn**).

As with LUTs, the system determines its advisory based on sensor data, which provides a seven-dimensional state vector as input of the network : (i) ρ – Distance between the ownship and the intruder. (ii) θ – Angle to the intruder relative to the ownship's heading direction. (iii) ψ – Intruder's heading angle relative to the ownship's heading. (iv) v_{own} – Speed of the ownship. (v) v_{int} – Speed of the intruder. (vi) τ – Time until loss of vertical separation. (vii) a_{prev} – Previous advisory issued by the system.

Based on these inputs, the system outputs one of the five possible horizontal advisories: COC, WL, WR, SL, SR.

The advisory decision-making process is handled by an array of 45 Deep Neural Networks. These networks are created by discretizing two input parameters: τ (Time to loss of vertical separation) and a_{prev} (Previous advisory).

This discretization results in 45 unique networks, each trained for a different scenario. Each DNN configuration has : 5 inputs (the remaining state dimensions), 5 outputs (corresponding to the advisories) and 6 hidden layers with 50 nodes each. They are fully connected architectures using ReLU activation functions.

By leveraging this DNN-based structure, ACAS-Xu significantly reduces memory requirements while maintaining high-speed, real-time decision-making for collision avoidance in autonomous aircraft and drones.

3.2 Alternative Neural Networks for ACAS-Xu

In addition to networks we discussed, we also have access to other neural networks that share the same architecture and design principles.

The new networks at our disposal have been trained specifically for scenarios where the intruder is on the same horizontal plane as the ownship, meaning $\tau = 0$ (no vertical separation).

These networks, while designed to perform the same collision avoidance function, differ in their training methodologies and underlying properties. The available networks are:

- **Adversarial_networks (Ad_nn)** : A set of five trained, one for each LUT. These networks were trained with the goal of satisfying a specific property we will introduce (cf. [2]). The learning strategy of these networks was focusing on training the models around the points of the LUTs.
- **Corners_networks (Co_nn)** : Another set of five trained networks, one for each LUT. These networks were trained with the same objective of satisfying a specific property like the Adversarial networks. But the learning strategy was different and was focusing on training the models specifically on the points of the LUTs.

4 Analysis of Unsafe Regions of Neural Network for ACAS-Xu

To ensure reliable operations of the ACAS-Xu system when replacing LUTs with neural network approximations, it is essential to identify and analyze regions where the network behavior may deviate from the expected safety logic. Our approach follows a two-step process. First, a formal verification phase is performed using a specification derived from the reference LUT, which captures the expected behavior of the real system. This step enables the detection of critical or potentially unsafe regions in the input space.

In the second step, a focused analysis is carried out on the LUT points corresponding to these critical regions to better characterize the deviations and assess their operational impact. This combined formal and behavioral analysis provides a comprehensive understanding of where and why the neural approximation may fail, guiding the application of appropriate mitigation strategies.

4.1 Introducing a Specification for ACAS-Xu

In the context of our framework, the first step toward identifying unsafe regions is the construction of a specification that captures the expected behavior of the ACAS-Xu system. This specification combines domain-specific properties, derived from the reference LUT, with behavioral requirements reflecting how the neural approximations should respond in equivalent operational conditions. Rather than only verifying whether the networks reproduce LUT outputs, this specification formalizes high-level behavioral expectations, checking that the system provides coherent advisories in representative encounter scenarios.

Some domain-specific properties have already been proposed in the literature, particularly through the use of formal verification tools such as Reluplex to assess neural network behavior [10]. However, these properties often address isolated cases and lack a structured organization, making it difficult to evaluate the network's overall coverage of the reference LUT and to pinpoint critical regions within the operational domain.

To overcome this limitation, we introduce a structured classification framework that categorizes the system's expected behaviors according to the type and complexity of encounter scenarios. The proposed categories include:

- Nominal Scenarios: Cases where the system should maintain a COC advisory.
- Single-Intruder Avoidance: Scenarios where one intruder approaches, and the system must provide a timely and appropriate avoidance maneuver.

- Multi-Intruder Conflicts: Situations with multiple threats where the system should issue a coherent and safe resolution.

This classification provides a systematic framework for analyzing the neural approximation's behavior under diverse operational conditions. It enables the identification of unsafe or weakly defined regions by a more comprehensive and global verification process, checking that neural networks not only approximate the LUT accurately but also behave consistently across a certain range of encounters situations.

It should be noted that our analysis is limited to the horizontal case, where the vertical separation between our aircraft and the intruders is zero ($\tau = 0$). Therefore, the following set of properties covers a broad range of possible flight scenarios on such a plane involving two or more aircraft. While the properties 2, 4, 9, 10 and 11 have been proposed by ourselves, the others are taken from [10].

Property 1. *For a far away intruder, the network advises COC.*
Input constraints : 36000 $\leq \rho \leq$ 60760, 0.7 $\leq \theta \leq$ 3.141592, -3.141592 $\leq \psi \leq$-3.141592 + 0.01, 900 $\leq v_{own} \leq$ 1200, 600 $\leq v_{int} \leq$ 1200
Desired Output : COC

Property 2. *If the intruder is approaching the ownship from behind at a lower speed, the network will advise COC.*
Input constraints : 55000 $\leq \rho \leq$ 10000, -3.1416 $\leq \theta \leq$-3.1416 + 0.01, -0.06 $\leq \psi \leq$ 0.06, 800 $\leq v_{own} \leq$ 1200, 0.0 $\leq v_{int} \leq$ 300
Desired Output : COC

Property 3. *If the intruder is sufficiently far away, the network advises COC.*
Input constraints : 12000 $\leq \rho \leq$ 62000, 0.7 $\leq \theta \leq$ 3.141592 *or* -3.1416 $\leq \theta \leq$-0.7, -3.141− $\leq \psi \leq$-3.141592 + 0.005
Desired Output : COC

Property 4. *If the intruder is approaching the ownship from behind and is going faster, the network will NOT advise COC.*
Input constraints : 55000 $\leq \rho \leq$ 10000, -3.1416 $\leq \theta \leq$-3.1416 + 0.01, -0.06 $\leq \psi \leq$ 0.06, 100 $\leq v_{own} \leq$ 300, 800 $\leq v_{int} \leq$ 1200
Desired Output : not COC

Property 5. *If the intruder is directly ahead and is moving towards the ownship, the score of a COC advisory is NOT the minimal score.*
Input constraints : 1500 $\leq \rho \leq$ 1800, -0.06 $\leq \theta \leq$ 0.06, $\psi \geq$ 3.10, $v_{own} \geq$ 980, $v_{int} \geq$ 960
Desired Output : not COC

Property 6. *If the intruder is directly ahead and is moving away from the ownship but at a lower speed than that of the ownship, the score for COC will NOT be minimal.*
Input constraints : 1500 $\leq \rho \leq$ 1800, -0.06 $\leq \theta \leq$ 0.06, ψ = 0.0, $v_{own} \geq$ 1000, 700 $\leq v_{int} \leq$ 800
Desired Output : not COC

Property 7. *If the intruder is near and approaching from the left, the network advises SR.*
Input constraints : 250 $\leq \rho \leq$ 400, 0.2 $\leq \theta \leq$ 0.4, -3.1416 $\leq \psi \leq$-3.1416 + 0.01, 100 $\leq v_{own} \leq$ 400, 0.0 $\leq v_{int} \leq$ 400
Desired Output : SR

Property 8. *Even if the previous advisory was WR, the presence of a nearby intruder will cause the network to output a SL advisory instead.*
Input constraints : 2000 $\leq \rho \leq$ 7000, -0.4 $\leq \theta \leq$-0.14, -3.1416 $\leq \psi \leq$-3.1416 + 0.01, 100 $\leq v_{own} \leq$ 150, 0.0 $\leq v_{int} \leq$ 150
Desired Output : SL

Property 9. *If two intruders are sufficiently far away, the network advises COC.*
Input constraints for Intruder 1 : 12000 $\leq \rho_1 \leq$ 62000, 0.7 $\leq \theta_1 \leq$ 3.141592 *OR* $-3.141592 \leq \theta_1 \leq -0.7$, $-3.141592 \leq \psi_1 \leq -3.141592 +$ 0.005, 100 $\leq v_{own-1} \leq$ 1200, 0.0 $\leq v_{int-1} \leq$ 1200
Input constraints for Intruder 2 : 12000 $\leq \rho_2 \leq$ 62000, 0.7 $\leq \theta_2 \leq$ 3.141592 *OR* $-3.141592 \leq \theta_2 \leq -0.7$, $-3.141592 \leq \psi_2 \leq -3.141592 +$ 0.005 100.0 $\leq v_{own-2} \leq$ 1200.0, 0.0 $\leq v_{int-2} \leq$ 1200.0
Desired Output : COC

Property 10. *If the intruder 1 is directly ahead and moving towards the ownship, and the intruder 2 is approaching from the left, the network advises a right turn (WR or SR).*
Input constraints for Intruder 1 : 1500.0 $\leq \rho_1 \leq$ 1800.0, $-0.06 \leq \theta_1 \leq$ 0.06, 3.10 $\leq \psi_1$, 300.0 $\leq v_{own-1} \leq$ 600.0, 300.0 $\leq v_{int-1} \leq$ 600.0
Input constraints for Intruder 2 : 1500 $\leq \rho_2 \leq$ 1800, 0.2 $\leq \theta_2 \leq$ 0.4, $-3.141592 \leq \psi_2 \leq -3.141592 +$ 0.005 300.0 $\leq v_{own-2} \leq$ 600.0, 200.0 $\leq v_{int-2} \leq$ 600.0
Desired Output : WR or SR

Property 11. *If the intruder 1 is directly ahead and moving towards the ownship, and the intruder 2 is approaching from the behind, the system will never advises a COC.*
Input constraints for Intruder 1 : 2000.0 $\leq \rho_1 \leq$ 22000.0, $-0.06 \leq \theta_1 \leq$ 0.06, 3.10 $\leq \psi_1$, 300.0 $\leq v_{own-1} \leq$ 600.0, 300.0 $\leq v_{int-1} \leq$ 600.0
Input constraints for Intruder 2 : 2000 $\leq \rho_2 \leq$ 22000, 0.2 $\leq \theta_2 \leq$ 0.4, $-0.06 \leq \psi_2 \leq$ 0.06 300.0 $\leq v_{own-2} \leq$ 600.0, 200.0 $\leq v_{int-2} \leq$ 600.0
Desired Output : not COC

The previously defined properties can be categorized into three distinct groups, each reflecting different operational expectations for the ACAS-X system.

- The first group (Properties 1 to 3) consists of scenarios where no avoidance maneuver is required. In these cases, the system should recognize the absence of a collision threat and correctly return COC as its advisory. Ensuring that ACAS-Xu consistently maintains COC in such situations is crucial to prevent unnecessary maneuvers that could disrupt flight operations or create inefficiencies. An example of a scenario from this group is given in Figure 5 : the intruder is approaching from behind at a lower speed, the network must advise COC.

- The second group (Properties 4 to 8) includes scenarios where the system must actively recommend an avoidance maneuver. In these cases, returning COC would indicate a

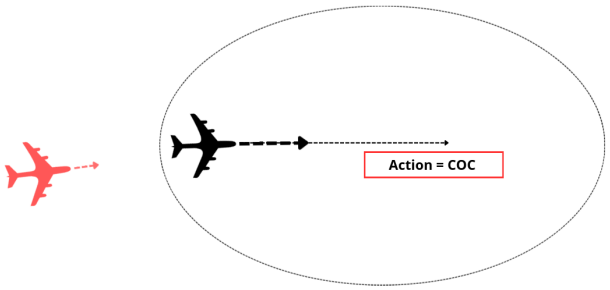

Figure 5: Group 1 Scenario

failure to respond appropriately to a potential collision risk. Instead, the system is expected to issue a specific advisory, directing the drone to adjust its trajectory to maintain safe separation from other aircraft. Proper validation of this group is essential to confirm that ACAS-Xu can reliably detect and react to hazardous encounters, ensuring robust mid-air collision prevention. An example of a scenario from this group is given in Figure 6 : the intruder is near and approaching from the left, the network must advise SR.

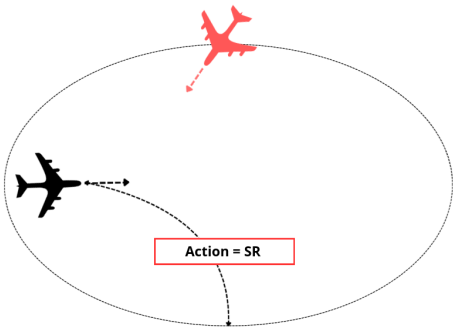

Figure 6: Group 2 Scenario

- The third group (Properties 9 to 11) includes scenarios involving multiple simultaneous threats. In these cases, the system must determine the appropriate avoidance maneuver to prevent collisions with any of the intruding aircraft. These three properties focus on situations with two intruders only. To remain consistent with the assumptions made in the previous sections, we restrict our analysis to the horizontal plane: the own aircraft and both intruders are assumed to be at the same altitude. An illustrative example from this group is shown in Figure 7: one intruder approaches head-on, while another approaches from the left. In this situation, the system should recommend a right turn (WR or SR).

This classification helps to structure the evaluation of ACAS-X's decision-making capabilities, distinguishing between situations where stability (holding COC) is desirable and those where proactive avoidance is necessary, while structuring more complex scenarios involving several threats to avoid, scenarios which will become

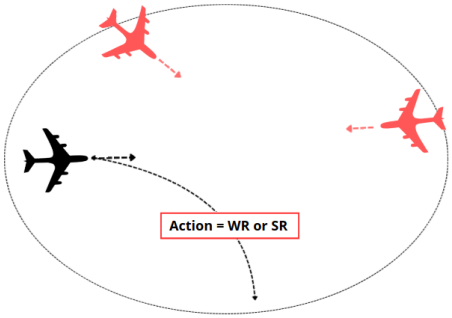

Figure 7: Group 3 Scenario

increasingly common with drones. By analyzing the system's responses within these three categories, we can better find the critical situations and configuration where the neural networks need compensation.

4.2 Verification Results

To evaluate the reliability of the neural network-based ACAS-Xu system, we conducted a series of experiments using the previously defined domain-specific properties. These tests were performed on the different neural network architectures available, including: J_nn [9], Ad_nn and Co_nn.

We use the CAISAR platform [7] to express our properties. This platform give us the possibility to use several solvers to execute the formal verification. Among the ones available, we choose to use Marabou [11] and Alpha-Beta-CROWN [17, 19, 20].

Marabou is a verification tool designed for analyzing neural networks, particularly deep learning models used in safety-critical systems. It is an extension of Reluplex [10], optimized for efficiency and scalability when verifying properties of networks with ReLU activation functions. Each property defines specific constraints on the flight state variables, such as distance or relative angle or speed, which describe particular encounter scenarios. These constraints are expressed as input intervals and are propagated through the neural networks by Marabou to formally assess whether the network's output satisfies the expected advisory behavior (Interval Bound Propagation).

In addition to Marabou, we also employ the Alpha-Beta-CROWN verifier [17, 19, 20], implemented through the AutoLiRPA framework [18]. Alpha-Beta-CROWN is a state-of-the-art neural network verification tool that combines linear relaxation-based bound propagation with branch-and-bound search to achieve both scalability and tightness of verification bounds. Unlike purely symbolic methods, it leverages efficient numerical optimization to reason about network behavior, making it particularly suitable for large and complex architectures. By expressing and executing verification tasks through the CAISAR platform, we can efficiently apply Alpha-Beta-CROWN to our networks, complementing Marabou with a method capable of handling non-linear activations and providing sound certified bounds. Due to its efficiency, Alpha-Beta-CROWN also enables the verification of Multi-Intruder properties, which

would be challenging to assess with other solvers. This integration strengthens our verification workflow, ensuring greater confidence in the safe and reliable operation of the ACAS-Xu system.

In every cases, if the network fails to meet the property within the defined input constraints, Marabou or Alpha-Beta-CROWN generate a concrete counterexample: a specific input state for which the network produces an invalid advisory. These counterexamples represent valid and actionable test cases, which can be cross-checked against the original system to understand the nature of the violation.

In parallel, the same verification process was applied to the original ACAS-Xu LUTs in order to ensure that the defined properties were fully consistent with the reference system's behavior. For each property, all LUT entries whose input parameters satisfied the corresponding constraints were extracted, and the associated advisories were compared against the expected outcomes defined by the property. This exhaustive analysis confirmed that the specification accurately reflects the intended decision logic of ACAS-Xu, rather than introducing any external or artificial assumptions.

All properties were successfully validated on the LUTs, demonstrating that they are well-founded representations of the system's true operational behavior. Consequently, any property violation detected in a neural network can be confidently interpreted as a genuine deviation from the certified ACAS-Xu logic.

We present in Table 1 the results of the verification of each property from our specification on the three sets of neural networks.

The verification was performed on a workstation equipped with an Intel Core i7 processor (5.2 GHz, 32 GB RAM). The average computation time ranged from 30 minutes to 1 hour for properties in Groups 1 and 2 on one neural network, while Group 3—which involves multi-intruder scenarios—required 2 to 3 hours per property on one neural network due to the increased complexity of the corresponding constraints.

Table 1: Testing Domain-Specific Properties on Different Neural Networks

P	J_nn	Ad_nn	Co_nn	LUT
P_6	Valid	Valid	Valid	Valid
P_{10}	Valid	Valid	Valid	Valid
P_{11}	Valid	Valid	Valid	Valid
P_3	Valid	Valid	Valid	Valid
P_4	Invalid*	Invalid	Invalid	Valid
P_5	Invalid*	Invalid*	Invalid*	Valid
P_9	Invalid	Valid	Invalid	Valid
P_{12}	Invalid*	Invalid	Valid	Valid
P_{14}	Invalid	-	-	Valid
P_{15}	Invalid	-	-	Valid
P_{16}	Invalid	-	-	Valid

Invalid* = Invalid with at least one network but not all 5.

All properties from Group 1 (Properties 1-3) were successfully validated, confirming that the networks consistently provide a Clear-of-Conflict (COC) advisory when required, in single intruder scenarios.

In contrast, all networks encounter difficulties with the Group 2 properties (Properties 4-8), which correspond to situations where the system must select a specific avoidance maneuver rather than maintaining a COC state. These scenarios are inherently more demanding, as they require a precise and consistent decision under asymmetric or time-critical conditions. The observed inconsistencies reveal specific regions of weakness, typically grouping the LUT points for which the minimum action associated is a specific avoidance advisory other than COC.

The challenge becomes even more significant when dealing with multiple simultaneous threats. Only the J_nn network was evaluated on the Group 3 properties (Properties 9–11), since the other two network sets (Ad_nn and Co_nn) were trained for classification tasks — i.e., to output the most likely advisory without considering the associated score or confidence value. In contrast, the Julian network was trained following the score of each possible action, theoretically enabling it to reason over multiple threats. Nevertheless, it still failed to satisfy any of the multi-intruder properties.

A particularly insightful observation arises from these tests: the multi-intruder scenarios include configurations that, when considered individually, correspond to validated LUT points. In other words, the networks behave correctly for each intruder taken separately, yet fail when both are active simultaneously. This demonstrates that current approximations are unable to capture the interaction dynamics between multiple threats — a limitation that cannot be corrected simply by applying a safety net, as it is proposed in [3], since the error lies in the network's decision structure itself rather than in localized approximation gaps.

Within the proposed framework, these results provide valuable information for detecting and localizing unsafe regions. By combining property-based verification with behavioral classification, we can identify the conditions under which the networks deviate from the certified logic and understand the nature of those deviations — whether they stem from unstable decision boundaries, conflicting score evaluations, or an inability to integrate multiple threat assessments. Consequently, this approach enables a targeted definition of mitigation zones.

Indeed, the main weakness identified across the three network sets corresponds to scenarios requiring a specific avoidance maneuver, such as advising a left or right turn. The detection of these unsafe regions remains, however, an approximate localization, as the current analysis does not yet quantify the proportion of correct versus incorrect advisories within each region. To achieve a more precise understanding of the system's behavior, a focused analysis must therefore be performed on the LUT points corresponding to these critical areas.

4.3 Focused Analysis on the LUT Points

A more detailed analysis was conducted on the LUT points corresponding to the scenarios covered by Group 2 of our specification (Properties 4–8), which represent situations where the system must issue a specific avoidance maneuver.

It is important to highlight that over 90% of the flight states enumerated in the ACAS-Xu LUTs result in a COC advisory. These are typically the states we do not want to consider. According to our results, from Table 1, these flight states are well manage by the neural networks (see the results on Properties 1 to 3).

To better localize and characterize the unsafe regions, we restricted our analysis to the subset of LUT entries that do not produce a COC advisory. This subset encompasses the most safety-critical and decision-sensitive situations, where the system must choose and maintain the appropriate avoidance maneuver, precisely the conditions under which the networks have shown the greatest variability and inconsistency.

Table 2: Network Conformaity on Non-COC LUT Points

LUT	J_nn	Ad_nn	Co_nn
a_prev = COC	57.44%	50.79%	84.79%
a_prev = WL	68.53%	56.36%	87.74%
a_prev = WR	69.95%	55.58%	82.85%
a_prev = SL	59.87%	53.66%	87.23%
a_prev = SR	59.58%	51.61%	88.70%

The results of this focused analysis are presented in Table 2. A significant discrepancy in accuracy can be observed between the networks and the reference LUT. These findings are consistent with the trends highlighted in the previous subsection and in Table 1. Yet, it is worth noting that the accuracy values reported in Table 2 correspond to only about 10% of the LUT, since the vast majority of entries lead to a COC advisory and were excluded from this analysis. When these results are scaled to the entire LUT, the overall accuracy levels are consistent with the global conformity rates of approximately 96–98% observed in [2].

Beyond confirming the existence of unsafe regions, this analysis provides a more detailed quantification of the LUT points associated with incorrect advisories, thereby allowing the precise identification of the zones where mitigation means should be applied.

4.4 Towards a Safety Net

By applying the previous analysis approach, we are able to construct a reduced LUT composed only of the reference entries where the neural networks fail to produce the correct advisory. During system execution, and particularly in avoidance situations, the current flight state can be checked against this reduced LUT: if it matches one of the identified critical states, the system retrieves the advisory directly from this LUT; otherwise, the neural network output is used. This reduced and safety-focused LUT thus acts as a Safety Net for the ACAS-Xu system.

Building such a Safety Net was the main objective of [3]. However, our contribution provides an efficient and systematic method for identifying the precise points that must be included within it. Through the strategy presented in Section 4, we focus exclusively on the most relevant subset of the LUT — the non-COC cases — while safely excluding nominal scenarios already validated in earlier stages. This selective exploration significantly reduces the verification effort while concentrating on the most safety-critical regions of the decision space.

Unfortunately, as discussed in Subsection 4.2, this Safety Net strategy alone is insufficient to handle multi-intruder scenarios. Additional or complementary mitigation techniques must therefore be investigated to ensure safe and consistent behavior under such complex conditions.

5 Related Works

The verification of neural networks used in safety-critical systems such as ACAS-Xu has been the subject of several recent studies.

Authors of [10] introduced Reluplex, one of the first SMT-based solvers capable of formally verifying properties of deep neural networks. Their work focused specifically on the ACAS-Xu collision avoidance system and demonstrated that formal tools could successfully prove safety-related properties, such as advisory consistency under specific encounter conditions. However, the verified properties may be defined directly on the network outputs and not structured according to operational scenarios, limiting the ability to analyze behavioral coverage or to localize weaknesses in the approximation. Moreover, their verification framework was restricted to single-intruder situations, without considering multiple simultaneous threats.

Later, [2] proposed a property-based verification approach to assess the fidelity of neural networks trained to approximate certified controllers such as ACAS-Xu. The proposed property ensures that, for every input region, the neural controller provides the same advisory as the reference table-based controller. This formalizes the notion of functional equivalence between the two decision policies and can be verified using SMT or MILP solvers within the SAM framework. While this method provides a strong formal guarantee of conformity, it remains focused on point-wise equivalence and does not explicitly account for the operational context in which advisories are generated.

More recently, authors of [3] introduced a certifiable hybrid architecture for ACAS-Xu, combining NN-based advisory generation with a Safety Net mechanism that reverts to LUT decisions in unverified regions. Using verification tools such as DeepPoly and Reluplex, they achieved a substantial memory reduction (from 4 GB to a few MB) while maintaining formal safety assurances. Their work also integrates an assurance case framework aligned with emerging certification standards (EUROCAE WG-114 [5]).

By leveraging a structured behavioral specification derived from the ACAS-Xu logic, our framework provides a more efficient way to construct a targeted Safety Net, focusing on the most safety-critical and operationally relevant scenarios. This enables a finer understanding of model weaknesses and a more practical integration of hybrid architectures in future certification workflows.

Beyond these static verification approaches, closed-loop analyses have been developed to evaluate neural network controllers in conjunction with aircraft dynamics. In particular, star set reachability methods have been proposed to compute over-approximations of all reachable states, offering formal guarantees of safety over continuous trajectories [14, 15]. While these methods provide strong theoretical assurances, they remain computationally demanding and mainly focus on trajectory reachability. In contrast, our work introduces a lightweight behavioral analysis framework that leverages structured properties to systematically assess ACAS-Xu neural approximations across a wide range of operational contexts, including complex multi-intruder encounters.

Authors of [1] further extended this direction by introducing a closed-loop verification approach for ACAS-Xu, combining state quantization with backreachability analysis to reason formally about system-level safety. Unlike most open-loop studies focusing on isolated network behavior, their method evaluates the complete neural network control system (NNCS), accounting for both dynamics and advisory updates over time. Their results demonstrated that even under ideal assumptions, early ACAS-Xu neural compressions could lead to unsafe behaviors, exposing collisions caused by incorrect advisories. This work marked an important step toward realistic, system-level verification.

Building on this perspective, our approach shifts the focus from proving absolute safety to detecting and characterizing weakness regions in neural approximations. While [1] identify specific unsafe trajectories in closed-loop execution, our framework generalizes this diagnostic capability by relying on domain-specific behavioral properties derived from the ACAS-Xu LUT. This enables systematic localization of unsafe regions and supports the design of targeted mitigation strategies, such as the construction of efficient Safety Nets.

Building upon these foundations, and particularly the hybrid approach of [3], our work proposes a method for analyzing and understanding the weaknesses of NN-based ACAS-Xu architectures. Through a hierarchical classification of behavioral properties—covering nominal, single-intruder, and multi-intruder avoidance—we enable a structured exploration of the system's decision space. This allows not only to detect the conditions under which networks deviate from the certified logic but also to identify where mitigation measures should be applied, ensuring robustness even in complex multi-threat configurations that remain challenging for existing verification frameworks.

6 Conclusion and Discussion

This work introduced a framework for identifying weaknesses in neural network approximations of the ACAS-Xu collision avoidance system. Rather than focusing solely on formal verification, the proposed approach combines property-based behavioral analysis with targeted exploration of the LUT, allowing a systematic localization of unsafe regions where the networks fail to reproduce the certified decision logic.

By defining a structured specification composed of domain-specific properties—covering nominal, single-intruder, and multi-intruder scenarios—we demonstrated how the approach can reveal the conditions under which the networks deviate from the reference behavior. Our focused analysis on the non-COC LUT points provided a detailed quantification of these deviations and guided the construction of an efficient reduced LUT, forming the basis of a Safety Net that can be used to ensure safe advisories during runtime.

However, this mitigation strategy remains insufficient for multi-intruder situations, where network limitations stem from the underlying decision representation rather than local approximation errors.

A key challenge in addressing multi-intruder situations lies in the representation and stability of action scores. To correctly reason over multiple threats, the system must retain the numerical scores associated with each advisory, rather than directly outputting a discrete class. This scoring approach allows the selection of the globally optimal action, but it also makes the model highly sensitive to small variations in network outputs. Even a minor numerical deviation between predicted scores can lead to an incorrect ranking and, consequently, an invalid advisory. Therefore, extending neural approximations to multi-intruder encounters requires not only architectural robustness but also improved control of score consistency and numerical precision.

Our structured analysis therefore provides deeper insight into the limitations of NN-based approximations and highlights areas where new mitigation strategies and architectures capable of handling complex multi-threat interactions are required.

While the proposed specification provides a solid foundation for behavioral analysis, its current scope does not guarantee complete coverage of the ACAS-Xu operational domain. The defined properties focus primarily on representative and safety-critical scenarios, but they may not encompass all possible configurations within the LUT. As a result, some regions of the decision space could remain unexplored, potentially hiding subtle weaknesses of the neural approximations.

To address this limitation, the specification could be expanded to include a broader range of flight scenarios, particularly those involving more complex or coordinated behaviors between aircraft. Introducing cooperative flights properties, for instance, would enable the evaluation of how multiple ACAS-Xu systems interact and respond jointly to conflict situations. Such extensions would not only improve the completeness of the verification process but also strengthen the detection of potential weaknesses in more realistic

and dynamically coupled operational environments.

Overall, the proposed framework offers a practical and extensible methodology for analyzing, understanding, and improving neural approximations in safety-critical systems. This methodology could be replicate for any system, featuring look-up-tables, approximate by neural networks.

References

[1] Stanley Bak and Hoang-Dung Tran. 2022. Neural network compression of ACAS Xu early prototype is unsafe: closed-loop verification through quantized state backreachability. In *NASA Formal Methods Symposium (NFM 2022)*. Springer Nature, Cham, Switzerland. doi:10.1007/978-3-031-06773-0_3.

[2] Arthur Clavière. 2023. *Safety verification of neural network based systems using formal methods*. Ph.D. Dissertation. http://www.theses.fr/2023ESAE0041/document. 2023ESAE0041.

[3] Mathieu Damour et al. 2021. Towards certification of a reduced footprint ACAS-Xu system: a hybrid ML-based solution. *Lecture Notes in Computer Science (LNCS)*, 12852, 31–43. doi:10.1007/978-3-030-83903-1_3.

[4] EUROCAE. 2020. ED-275: Minimum Operational Performance Standards for Airborne Collision Avoidance System Xu (ACAS-Xu). Tech. rep. Legal notice: Licensed to ONERA. EUROCAE, (Dec. 2020). https://www.eurocae.net/documents-and-projects/documents/ed-275/.

[5] EUROCAE WG-114/SAE Joint Group. 2021. Certification/Approval of Aeronautical Systems Based on AI. Tech. rep. Ongoing standardization effort. EUROCAE and SAE International.

[6] Eurocontrol. 2022. ACAS guide. https://www.eurocontrol.int/system/acas.

[7] Julien Girard-Satabin, Michele Alberti, François Bobot, Zakaria Chihani, and Augustin Lemesle. 2022. CAISAR: a platform for characterizing artificial intelligence safety and robustness. In *AISafety* (CEUR-Workshop Proceedings). Vienne, Austria, (July 2022). https://hal.archives-ouvertes.fr/hal-03687211.

[8] Arnaud Gotlieb. 2012. Tcas software verification using constraint programming. *The Knowledge Engineering Review*, 27, 3, 343–360. doi:10.1017/S0269888912000075.

[9] Kyle D. Julian, Mykel J. Kochenderfer, and Michael P. Owen. 2019. Deep neural network compression for aircraft collision avoidance systems. *Journal of Guidance, Control, and Dynamics*, 42, 3, 598–608. doi:10.2514/1.G003724.

[10] Guy Katz, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. 2017. Reluplex: an efficient SMT solver for verifying deep neural networks. In *Computer Aided Verification*. Rupak Majumdar and Viktor Kunčak, (Eds.) Springer International Publishing, Cham, 97–117. ISBN: 978-3-319-63387-9. doi:https://doi.org/10.1007/978-3-319-63387-9_5.

[11] Guy Katz et al. 2019. The Marabou framework for verification and analysis of deep neural networks. In *Computer Aided Verification*. Isil Dillig and Serdar Tasiran, (Eds.) Springer International Publishing, Cham, 443–452. ISBN: 978-3-030-25540-4.

[12] Mykel J. Kochenderfer and James P. Chryssanthacopoulos. 2011. Robust Airborne Collision Avoidance through Dynamic Programming. Project Report ATC-371 ATC-371. Prepared for the Federal Aviation Administration. MIT Lincoln Laboratory, Lexington, MA, (Jan. 2011). https://apps.dtic.mil/sti/pdfs/ADA537522.pdf.

[13] Mykel J. Kochenderfer, Jessica E. Holland, and James P. Chryssanthacopoulos. 2012. Next-generation airborne collision avoidance system. *Lincoln Laboratory Journal*, 19, 1, 17–33. https://www.ll.mit.edu/sites/default/files/page/doc/2018-05/19_1_2_Kochenderfer.pdf.

[14] Joaquin Lopez, Taylor Johnson, Hoang-Dung Tran, and Taylor T. Johnson. 2021. Verification of neural network compression of ACAS Xu lookup tables with star set reachability. In *Assured Autonomy Workshop, Assured Autonomy Center of Excellence*. Accessed 2025-10-14. https://assured-autonomy.org/publications/verification-neural-network-compression-acas-xu-lookup-tables-star-set-reachability.

[15] Sina Sheikhi and Stanley Bak. 2023. Closed-loop ACAS Xu neural network verification. In *10th International Workshop on Applied Verification for Continuous and Hybrid Systems (ARCH 2023)*. Proceedings of ARCH23. EasyChair. https://easychair.org/publications/paper/mmSLW.

[16] Jun Tang, Fen Zhu, Wan Yu, and Songyang Lao. 2018. Collision avoidance performance improvement for tcas in multi-aircraft situations based on state prediction. *Journal of System Simulation*, 30, 12, 4703–4711. doi:10.16182/j.issn1004731x.joss.201812026.

[17] Shiqi Wang, Huan Zhang, Kaidi Xu, Xue Lin, Suman Jana, Cho-Jui Hsieh, and Zico Kolter. 2021. Beta-CROWN: efficient bound propagation with per-neuron split constraints for neural network robustness verification. *Advances in Neural Information Processing Systems (NeurIPS)*, 34, 29909–29922. doi:10.48550/arXiv.2103.06624.

[18] Kaidi Xu, Zhouxing Shi, Huan Zhang, Yihan Wang, Kai-Wei Chang, Minlie Huang, Bhavya Kailkhura, Xue Lin, and Cho-Jui Hsieh. 2020. Automatic perturbation analysis for scalable certified robustness and beyond. In *Advances in Neural Information Processing Systems (NeurIPS 2020)*. auto_LiRPA: an automatic linear relaxation based perturbation analysis library. https://arxiv.org/abs/2002.12920.

[19] Kaidi Xu, Huan Zhang, Shiqi Wang, Yihan Wang, Suman Jana, Xue Lin, and Cho-Jui Hsieh. 2022. Fast and complete: enabling complete neural network verification with rapid and massively parallel incomplete verifiers. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 28168–28183. doi:10.48550/arXiv.2205.14224.

[20] Huan Zhang, Tsui-Wei Weng, Pin-Yu Chen, Cho-Jui Hsieh, and Luca Daniel. 2018. CROWN: efficient neural network robustness certification with general activation functions. *Proceedings of the 32nd Conference on Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/1811.00866.